

Backend onboarding

Este backend arranca como monolito modular. La unidad de despliegue es una sola API, pero el código se organiza por módulos de negocio para que después sea posible extraer partes si realmente aparece la necesidad.

Dominio inicial

De los documentos del proyecto, el sistema apunta a una plataforma de aprendizaje adaptativo para reducir la brecha entre formación secundaria y mercado laboral IT.

Actores iniciales:

- `student`: estudiante que realiza diagnóstico, learning path, prácticas y evaluaciones.
- `tutor`: humano que monitorea progreso, riesgo de abandono y evaluaciones críticas.
- `company_admin`: referente de empresa/oferta laboral que puede necesitar ver avances.
- `admin`: operación interna de la plataforma.

Por eso el primer módulo es `users`: no resuelve todo el dominio, pero permite registrar los actores sobre los que después cuelgan onboarding, diagnósticos, cursos, ofertas y seguimiento.

Flujo de trabajo local

Activar entorno:

```
source .venv/bin/activate
```

Levantar API:

```
uvicorn app.main:app --reload
```

Abrir documentación Swagger:

```
http://localhost:8000/docs
```

Crear usuario:

```
curl -X POST http://localhost:8000/users \  
-H "Content-Type: application/json" \  
-d
```

```
'{"first_name":"Ana","last_name":"Perez","email":"ana@example.com","role":"student"}'
```

Listar usuarios:

```
curl http://localhost:8000/users
```

Dónde poner cada cosa

`router.py` recibe HTTP, valida payloads y delega en servicios. Mantiene los endpoints finos y evita mezclar reglas de negocio con transporte.

`service.py` contiene reglas de negocio y decisiones de error. Por ejemplo: no permitir emails duplicados.

`repository.py` encapsula SQLAlchemy. Si mañana cambia una query, idealmente no toca el router.

`models.py` define tablas y columnas SQLAlchemy.

`schemas.py` define contratos Pydantic para requests y responses.

Decisiones MVP

- SQLAlchemy síncrono: más simple para empezar; FastAPI lo soporta bien.
- `create_all` al iniciar: aceptable para MVP sin Alembic.
- Sin Docker local: se puede correr directo con Python y PostgreSQL.
- Sin auth todavía: primero cerrar CRUD y flujos de dominio.
- Sin microservicios: módulos internos, una sola base y un solo deploy.

Próximo módulo sugerido

Después de `users`, el siguiente corte natural es `learning_profiles` u `onboarding`, con datos mínimos del estudiante:

- puesto objetivo
- intereses
- disponibilidad horaria
- nivel técnico inicial
- habilidades declaradas

Eso conecta directamente con la Fase 0/Fase 1 descrita en los documentos: diagnóstico de brecha y anclaje motivacional.